

MAVP: Map-Aware Visuomotor Policies for Mobile Manipulation

Jinhe Tang^{1,†}, Ruixiao Dai^{1,†}, and Weiming Zhi^{1,2,3,*}

¹School of Computer Science and ²Australian Center For Robotics, The University of Sydney, Australia

³College of Connected Computing, Vanderbilt University, TN, USA

[†]These authors contributed equally.

*Corresponding author: Weiming.Zhi@sydney.edu.au

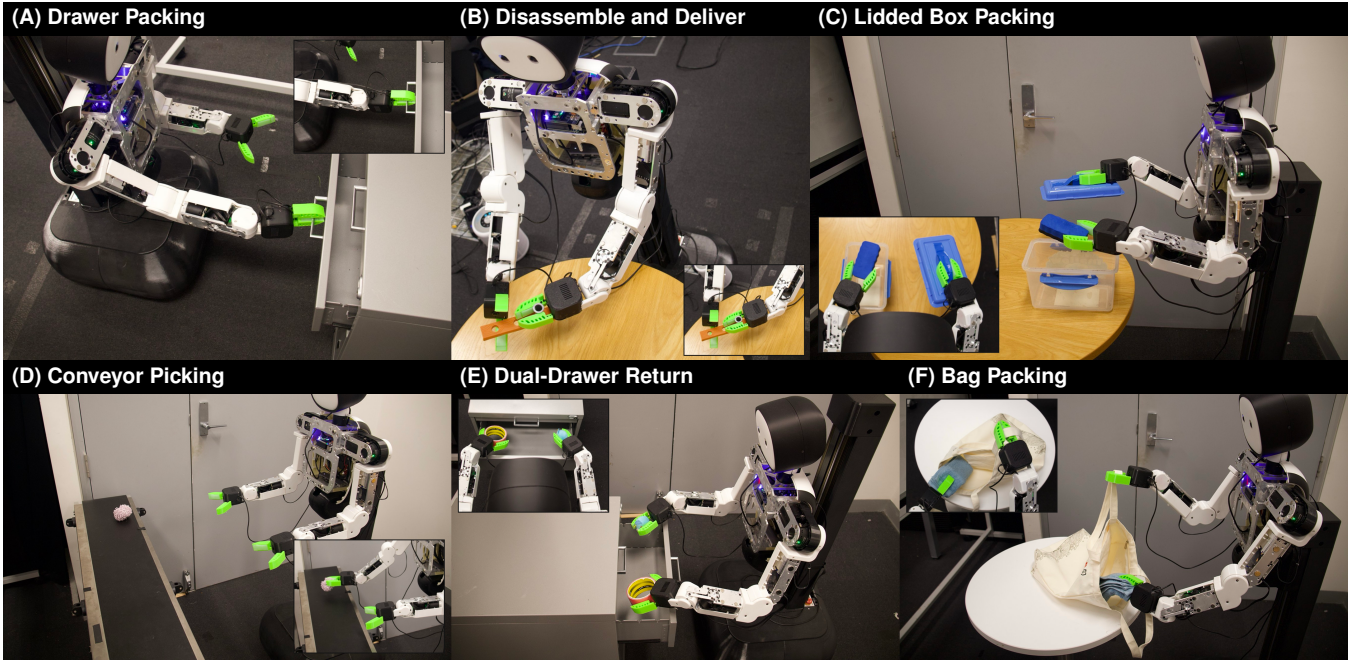


Fig. 1: We propose Map-Aware Visuomotor Policies (MAVP), evaluated on six real-world mobile manipulation tasks requiring coordinated base motion and manipulation across multiple workspace locations.

Abstract—Successful mobile manipulation requires coordinated base and arm motion while maintaining accurate spatial positioning. However, demonstration-trained policies can struggle to realise the intended base motion reliably, leading to spatial misalignment and subsequent manipulation failures. We present MAVP (Map-Aware Visuomotor Policies), a framework that improves execution reliability by predicting explicit base-pose targets and tracking them using localisation feedback. MAVP reconstructs a static map from teleoperated demonstrations and expresses demonstrated base trajectories in a shared map frame, providing consistent spatial supervision across demonstrations. At execution time, the policy receives RGB observations, joint states, and the robot’s current map-frame base pose, and jointly predicts target base poses, arm actions, and gripper actions. A low-level controller tracks the predicted base targets using feedforward motion and pose-error feedback, enabling correction of execution deviations. We additionally use pose-noise augmentation during training to improve robustness to errors in the policy’s pose input. Across six real-world manipulation tasks and three policy families, MAVP achieves higher task success rates than unanchored velocity control in all tasks.

I. INTRODUCTION

Mobile manipulation requires robots to coordinate base and arm motion across multiple locations. For example,

disassembling an object and delivering its parts requires the base to reach suitable poses for manipulation at successive locations. Base position and orientation affect both the arms’ reachable workspace and the camera viewpoints, so positioning errors can disrupt subsequent manipulation. Teleoperated demonstrations support learning coordinated base and arm behaviours [1], but reliable execution remains challenging.

Prior work explores representations and coordination for mobile manipulation. *Mobi- π* selects suitable base poses for a pretrained manipulation policy [2], while *HoMeR* combines learned end-effector pose targets with whole-body control [3]. Spatial Action Maps represent actions in a spatial map [4], and other policies use features from a learned 3D map [5].

Without an explicit base-pose target, a velocity-based policy controls base motion incrementally, causing small prediction and tracking errors to accumulate into base-pose drift over time. The policy must also infer its spatial state from visual and proprioceptive observations to continuously adjust the base motion. Pose-based targets instead provide an explicit spatial objective for feedback control. However, independently initialised coordinate frames across demonstrations can assign different coordinates to the same base pose in the workspace.



Fig. 2: Overview of MAVP: map localisation and visual odometry (VO) estimate the base pose in a shared static map reconstructed from masked demonstrations. The policy jointly predicts base-pose targets and arm/gripper actions; feedforward (FF) and pose-error feedback (FB) track the base targets (camera views are illustrative).

A shared map resolves this ambiguity by providing a consistent frame for demonstration supervision and target tracking.

We present **MAVP** (Map-Aware Visuomotor Policies), which expresses demonstrated base trajectories, current map-frame base-pose estimates, and predicted base targets in a shared map frame. The policy receives visual observations, joint states, and the estimated base pose, and jointly predicts base-pose targets, arm actions, and gripper actions. A low-level controller converts the base targets into velocity commands using feedforward motion and pose-error feedback, allowing execution deviations to be corrected without requiring the policy to predict each corrective velocity command.

MAVP constructs its spatial reference from teleoperated demonstrations, masking robot arms and moving objects before reconstructing a static scene map. During execution, visual localisation, camera calibration, and robot kinematics provide the base-pose estimate. To improve tolerance to imperfect localisation, we perturb the policy’s pose inputs during training while retaining the demonstrated action targets.

Our technical contributions include,

- **Shared map construction from demonstrations.** We construct a shared static map from teleoperated demonstrations by masking robot arms and dynamic regions before reconstruction. The map provides a common spatial reference for aligning demonstrations and expressing base states and action targets across episodes.
- **A map-aware policy learning and execution framework.** We introduce MAVP, which conditions the policy on the current map-frame base pose and jointly predicts base-pose targets, arm actions, and gripper actions. A feedforward-plus-feedback controller tracks the base targets, while pose-noise augmentation improves robustness to errors in the policy pose input.
- **Evaluation across tasks, policy families, and system components.** We rigorously evaluate and analyse the properties of MAVP on real-world tasks using different policy classes.

II. RELATED WORK

Mobile manipulation and spatial representations: Mobile ALOHA learns whole-body actions from teleoperation [1], M3 composes reinforcement-learned manipulation and navigation skills [6], and Mobi- π selects base poses for a pretrained manipulation policy [2]. UMI-on-Legs and HoMeR connect learned end-effector targets to whole-body controllers [3], [7]. HoMMI uses a gripper-centred frame for hand-eye policy observations and actions [8]. Spatial Diagrammatic Instructions express spatial objectives and constraints from sketches for mobile-base placement [9]. Other methods use point clouds or maps for geometric context [4], [5], [10].

MAVP learns base-pose targets in a shared map frame and tracks them with localisation feedback, while jointly predicting arm and gripper actions.

Action representation and imitation robustness: Automatic Waypoint Extraction (AWE) compresses demonstrations into spatial waypoints [11], while HYDRA combines sparse waypoints, dense actions, and action relabelling [12]. Dagger addresses policy-induced distribution shift through expert labels on learner-visited states [13], and DART perturbs demonstration execution to collect corrective behaviour [14]. MAVP addresses localisation uncertainty through pose-noise augmentation without collecting corrective demonstrations.

Mapping and localisation: Scene mapping and localisation methods reconstruct landmarks, match images, estimate camera poses, and mask dynamic regions [15]–[19]. Learned matchers such as SuperGlue and LightGlue use contextual feature relationships to estimate correspondences [20], [21]. Joint calibration and scene representation methods align reconstructed geometry with robot frames [22], [23]. In MAVP, established mapping and localisation techniques provide a shared scene reference for base-pose supervision and execution feedback.

III. MAVP: MAP-AWARE VISUOMOTOR POLICIES

MAVP uses a shared map as a common spatial reference for teleoperated demonstrations, policy learning, and closed-loop execution. As shown in Figure 2, the shared map is first reconstructed from the demonstrations while excluding non-static scene content. During deployment, live observations are

then matched against this map, together with visual odometry, to estimate the current base pose in the map frame. Finally, the estimated pose provides the spatial reference for the visuomotor policy, which predicts map-referenced base targets together with manipulation actions.

System setup: MAVP assumes that the mapped scene geometry remains static. In our setup, we use a dual-arm wheeled robot with calibrated head and wrist cameras. The head-mounted stereo RGB cameras provide map reconstruction and visual odometry; calibrated camera-to-base extrinsics transform camera-pose estimates into map-frame base poses.

A. Shared Map Construction

MAVP builds a static map offline from demonstrations. Robot-arm and dynamic-object regions are masked out so that the map is reconstructed from static scene features. Figure 3 summarises the static-scene filtering and shared map reconstruction pipeline, together with the resulting reconstructed map.

Reference Sampling and Static-Scene Filtering: MAVP uniformly samples stereo image pairs from the head-mounted cameras in each demonstration to construct the shared map $\mathcal{M}$. Let I_n^c denote the n -th sampled image from camera $c \in \{L, R\}$, where L and R denote the left and right stereo cameras. To exclude the robot from map reconstruction, we project the robot model into each image using the joint configuration, forward kinematics, and calibrated camera parameters, yielding a binary robot-arm mask A_n^c . Here, $A_n^c(u) = 1$ marks pixel u as part of a projected robot arm.

Dynamic scene regions can violate the cross-view consistency required for static map reconstruction. We therefore apply VGGT4D [19] independently to the image sequence from each head-camera view to separate dynamic elements from the static scene. VGGT4D extracts dynamic cues from VGGT’s global attention, aggregates them over a temporal window, and applies projection-gradient refinement to sharpen the resulting region boundaries. This yields a binary dynamic-region mask D_n^c for each sampled image, where $D_n^c(u) = 1$ indicates that pixel u is classified as dynamic.

We combine the robot-arm and dynamic-region masks to retain only regions used for static map reconstruction:

$$M_n^c(u) = (1 - A_n^c(u))(1 - D_n^c(u)). \quad (1)$$

Thus, $M_n^c(u) = 1$ indicates that pixel u is retained for map reconstruction, while robot-arm and dynamic regions are excluded. We denote the sampled images and their corresponding mapping masks by $\mathcal{I}_M = \{(I_n^c, M_n^c)\}_{n,c}$.

Shared Map Reconstruction: Using the retained regions in $\mathcal{I}_M$, MAVP extracts and matches visual features across sampled frames, demonstrations, and stereo views. COLMAP [15] performs two-view geometric verification to reject inconsistent matches, and then reconstructs a shared sparse map by registering the reference images and triangulating matched features into 3D landmarks. Bundle adjustment jointly refines the camera poses and landmark positions while keeping the calibrated camera intrinsics and stereo geometry fixed. The resulting map $\mathcal{M}$ contains the reference camera poses, 3D

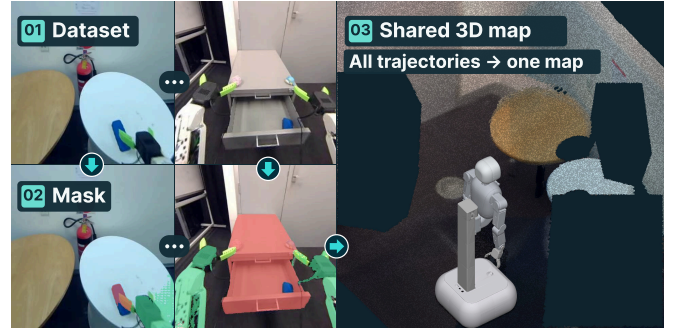


Fig. 3: Shared map reconstruction from stereo demonstrations after masking robot arms and dynamic regions.

landmarks, and their associated image features, which are subsequently used for visual localisation.

B. Map-Based Localisation

During execution, MAVP localises the head camera in the shared map and transforms the estimated camera pose into a map-frame base pose using the calibrated camera geometry and robot kinematics. The resulting base pose is used by the policy and feedback controller.

Visual Query Localisation At execution time, MAVP localises the head-camera observations against the shared static map. Features on the robot arm move with the robot and therefore cannot serve as stable references in the map. We apply the robot-arm masking procedure described in Sec. III-A to the current stereo images and exclude features within the projected robot regions from localisation.

MAVP first retrieves candidate reference images from the shared map based on visual similarity. SuperPoint [24] detects salient local keypoints in both the query and reference images and computes a descriptor for each keypoint. LightGlue [21] then compares these keypoints and descriptors across the image pair to identify geometrically consistent feature matches. Because the matched reference features are already associated with 3D landmarks in $\mathcal{M}$, each accepted image match links a 2D query keypoint to a corresponding 3D map point.

Perspective- n -Point (PnP) estimation with random sample consensus (RANSAC) recovers the head-camera pose from these 2D–3D matches. Pose estimates must meet inlier-count, reprojection-error, and stereo-consistency thresholds.

Map–Odometry Alignment and Base-Pose Estimation:

MAVP uses cuVSLAM stereo visual odometry [25] to continuously track the head-camera motion in a local coordinate frame. cuVSLAM recovers metric geometry from stereo correspondences and estimates relative camera motion by tracking features across frames, producing a continuous local pose trajectory. Whenever map-based localisation provides the camera pose in the shared map, we align the odometry trajectory using paired local and map-frame poses.

Let L , M , C , and B denote the local tracking, shared map, head-camera, and mobile-base frames. The rigid transformation ${}^X\mathbf{T}_{Y,t}$ maps coordinates from frame Y to frame X at time t . cuVSLAM provides ${}^L\mathbf{T}_{C,t}$, and map localisation provides ${}^M\mathbf{T}_{C,t}$ at the corresponding timestamp.

Using these two corresponding poses, we compute the alignment $\mathbf{T}_{\text{align},t}$ from the local tracking frame to the shared map frame as

$$\mathbf{T}_{\text{align},t} = {}^M\mathbf{T}_{C,t} ({}^L\mathbf{T}_{C,t})^{-1}. \quad (2)$$

Applying $\mathbf{T}_{\text{align},t}$ transforms the continuous cuVSLAM trajectory into the shared map frame. Subsequent map localisations update this alignment, with smoothing to limit pose jumps.

The mobile-base pose is then obtained from the aligned camera pose using the camera-to-base transformation:

$${}^M\mathbf{T}_{B,t} = \mathbf{T}_{\text{align},t}^{\text{smooth}L} \mathbf{T}_{C,t} ({}^B\mathbf{T}_{C,t})^{-1}. \quad (3)$$

Here, $\mathbf{T}_{\text{align},t}^{\text{smooth}}$ denotes the smoothed local-to-map alignment, and ${}^B\mathbf{T}_{C,t}$ represents the head-camera pose relative to the mobile base. The resulting ${}^M\mathbf{T}_{B,t}$ gives the base pose in the shared map frame. We extract its planar pose (x, y, ψ) , consisting of the map-frame position (x, y) and yaw ψ , for use as policy input and in feedback control.

C. Map-Aware Visuomotor Policies Learning

MAVP uses the shared map to provide a common spatial reference for the base-related components of the policy state and action. Let $\mathbf{s}_t$ collect the visuomotor observations available at time t , and let $\mathbf{b}_t = (x_t, y_t, \psi_t)$ denote the current base pose in the shared map frame obtained from Eq. 3. The policy π_θ , parameterised by θ , predicts a sequence of H future actions, where H is the action horizon:

$$\hat{\mathbf{A}}_{t:t+H-1} = \pi_\theta(\mathbf{s}_t, \mathbf{b}_t), \quad (4)$$

where $\hat{\mathbf{A}}_{t:t+H-1}$ denotes the predicted action sequence from time t to $t+H-1$. Action components depend on the task and robot interface; base motion is represented by target poses in the shared map frame. We refer to tracking these base-pose targets as *map-frame pose control*.

The policy is trained by imitation learning from teleoperated demonstrations. Let $\mathcal{D} = \{(\mathbf{s}_t, \mathbf{b}_t, \mathbf{A}_{t:t+H-1})\}$ denote the demonstration dataset. Imitation learning trains the policy to reproduce these demonstrated actions from the observed state by minimising

$$\mathcal{L}_{\text{IL}}(\theta) = \mathbb{E}_{\mathcal{D}} [\ell(\pi_\theta(\mathbf{s}_t, \mathbf{b}_t), \mathbf{A}_{t:t+H-1})], \quad (5)$$

where the expectation is over samples from $\mathcal{D}$ and ℓ denotes the policy-specific imitation-learning loss. Different policy families instantiate ℓ differently, while sharing the same objective of matching the demonstrated behaviour.

During execution, the current base pose is obtained from visual localisation and is therefore subject to estimation noise. To expose the policy to similar variations during training, we first estimate the localisation-noise distribution from offline localisation runs. We use the empirical standard deviations of planar position and yaw fluctuations to define a zero-mean Gaussian with covariance

$$\Sigma = \text{diag}(\sigma_x^2, \sigma_y^2, \sigma_\psi^2).$$

Here, σ_x , σ_y , and σ_ψ are the standard deviations of the noise in x , y , and yaw. For each training sample, we add an independent perturbation to the map-frame base pose:

$$\epsilon_t \sim \mathcal{N}(\mathbf{0}, \Sigma), \quad \tilde{\mathbf{b}}_t = \mathbf{b}_t + \epsilon_t. \quad (6)$$

Here, ϵ_t denotes the sampled localisation perturbation and $\tilde{\mathbf{b}}_t$ the perturbed base pose. The demonstrated action sequence is

kept unchanged, while the base-pose input $\mathbf{b}_t$ is replaced by $\tilde{\mathbf{b}}_t$. The augmented imitation-learning objective is therefore

$$\mathcal{L}_{\text{IL}}^{\text{aug}}(\theta) = \mathbb{E}_{\epsilon_t \sim \mathcal{N}(\mathbf{0}, \Sigma)} \left[\ell(\pi_\theta(\mathbf{s}_t, \tilde{\mathbf{b}}_t), \mathbf{A}_{t:t+H-1}) \right], \quad (7)$$

This augmentation encourages the policy to produce consistent actions under small localisation errors.

IV. EMPIRICAL EVALUATION

We evaluate MAVP on six real-world mobile manipulation tasks. We test whether a shared spatial reference improves execution, which mapping, policy, and control components contribute, and whether ongoing map localisation improves repeated task execution. We organise the evaluation around the following questions:

- Q1:** Does map-frame pose control improve task success compared with unanchored velocity and episode-initialised odometry pose control?
- Q2:** With the same base-pose input, how do map-frame pose targets compare with base velocities?
- Q3:** What is the contribution of explicitly providing the current map-frame base pose to the policy?
- Q4:** Does pose-noise augmentation improve tolerance to errors in the policy’s pose input?
- Q5:** How does pose-error feedback affect task success and base-target tracking?
- Q6:** Which map-construction components support complete maps and reliable held-out localisation?
- Q7:** Does MAVP improve task success across different visuomotor policy families?
- Q8:** Does ongoing map localisation improve repeated task execution compared with initial alignment only?

A. Common Experimental Setup

Physical setup:

Fig. 4: Configuration with one head-mounted and two wrist cameras.

The robot has a four-wheel omnidirectional base, a lifting and rotating torso, and two 7-DoF arms with independent grippers. Base commands specify planar velocity (v_x, v_y) and yaw rate. Arm commands specify joint angles and gripper states. The policy receives images from a head-mounted RGB camera and two wrist RGB cameras. Figure 4 shows the robot configuration and camera placements.

Figure 1 shows Drawer Packing, Disassemble and Deliver, Lidded Box Packing, Conveyor Picking, Dual-Drawer Return, and Bag Packing in panels A to F. Insets provide close-up or additional views. Table I gives the task sequences and success criteria.

Training setup: We instantiate MAVP with three visuomotor policy families: ACT, DP, and FM. The controlled ablations in Q1–Q5 and the ongoing-localisation experiment in Q8 use ACT; Q7 compares all three families. We train all policies for 30,000 optimisation steps using automatic mixed precision and eight data-loader workers. ACT uses a batch size of 32 and a learning rate of 3×10^{-5} , with one observation step and an action chunk of 30 steps. The DiT-based implementations [26] used for Diffusion Policy (DP)

TABLE I: Six mobile manipulation tasks; success requires completing every listed step.

Task	Description
Drawer Packing	Pick up the eraser, open the drawer, put the eraser and two toys inside, and close the drawer.
Disassemble and Deliver	Remove the peg, move to the box, and place all parts inside.
Conveyor Picking	Track and grasp an object on a moving conveyor.
Lidded Box Packing	Approach and open the box, place an object inside, and close the lid.
Bag Packing	Take a bag from the rack, put the table objects inside, and hang the bag back on the rack.
Dual-Drawer Return	Return the two objects outside the drawers to their corresponding drawers.

TABLE II: Success rates (%) for the control variants. Pose denotes the map-frame base-pose input; aug. denotes training pose-noise augmentation.

Method / variant	Disassemble and Deliver	Lidded Box Packing
Unanchored velocity	38%	44%
Odometry pose	16%	16%
Map-frame pose (w/o pose)	74%	58%
Velocity (w/ pose)	60%	64%
Map-frame pose (w/ pose)	84%	76%
Map-frame pose (w/ pose + aug.)	90%	88%

and Flow Matching (FM) use a batch size of 64 and a learning rate of 2×10^{-5} , with two observation steps, a prediction horizon of 32 steps, and 24 action steps.

Evaluation metrics: A demonstration or policy rollout succeeds when it completes every step of the corresponding task. Success rate is the percentage of successful rollouts. Each task has 50 teleoperated demonstrations, and each evaluated policy receives 50 rollouts per task.

B. Map-Frame Pose Control (Q1, Q2, and Q3)

Experimental setup: Table II compares six visuomotor policy variants on Disassemble and Deliver and Lidded Box Packing. All variants receive arm joint and gripper state. In variant labels, pose denotes the current map-frame base pose and aug. denotes pose-noise augmentation during training.

Unanchored velocity predicts base velocities without a map or base-pose input. *Odometry pose* predicts pose targets in a frame initialised at the start of each episode. Its controller uses live odometry to execute those targets. *Map-frame pose (w/ pose)* receives the current map-frame base pose and predicts targets in that frame.

Map-frame pose (w/o pose) removes the base-pose input while retaining map-based localisation and base-pose targets. *Velocity (w/ pose)* retains this input and predicts velocity commands. *Map-frame pose (w/ pose + aug.)* adds the training augmentation from Eq. 6. This perturbs the pose input, preserving images, joint/gripper states, and action targets.

Figure 5 uses samples from the demonstration datasets that train the two task policies. The position-label plots report median pairwise position gaps among nearby demonstration states when expressed in three coordinate frames: raw odometry (episode-local), aligned odometry (odometry rigidly transformed into the shared map frame via T_{align}), and map coordinates (absolute poses from visual localisation). The gap measures label inconsistency across demonstrations. The

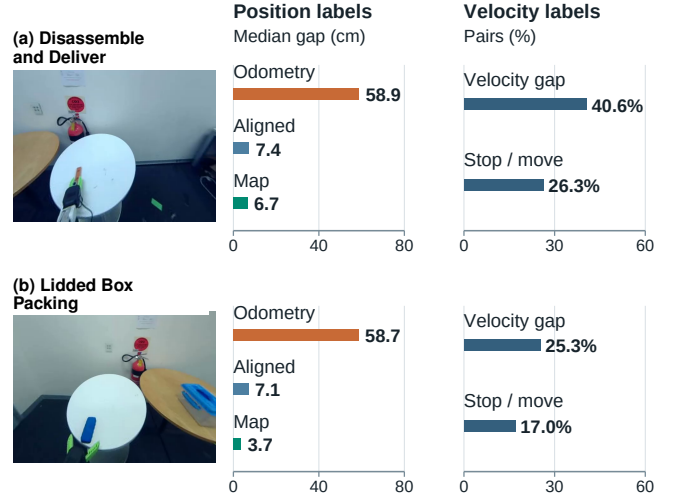


Fig. 5: Training-label variation for (a) Disassemble and Deliver and (b) Lidded Box Packing. Panels compare median position gaps across frames and fractions of nearby-state pairs with differing velocity or stop/move labels.

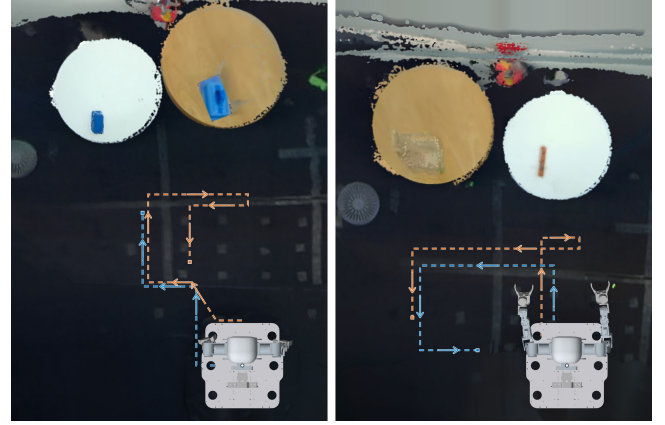


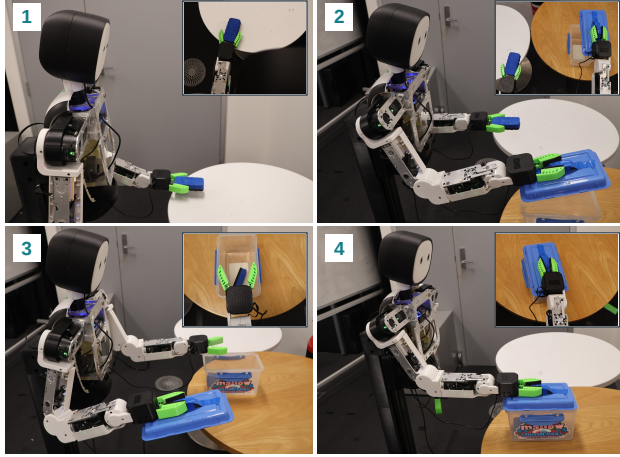
Fig. 6: Base trajectories for Lidded Box Packing (left) and Disassemble and Deliver (right). Velocity control (blue) stops early; map-frame pose control with training augmentation (yellow) continues toward task locations.

velocity-label plots compare target velocities at nearby robot states. Together, they characterise training-label variation.

Results: Table II shows complementary benefits from map-frame targets, explicit pose inputs, and training augmentation. The complete configuration performs best on both tasks, while the controlled comparisons below distinguish the roles of its components. For Q1, map-frame pose control improves task completion over both unanchored velocity and episode-initialised odometry pose control. Its spatial targets give the controller an objective to track in the same frame used for demonstration supervision, so execution can respond to deviations from a target instead of relying only on predicted velocity commands. Figure 6 contrasts velocity-controlled examples that stop short with augmented map-frame pose examples that continue towards manipulation locations; Figure 7 shows task progression.

For Q2, *Map-frame pose (w/ pose)* outperforms *Velocity (w/ pose)* on both tasks without training augmentation. Both policies receive the current map-frame pose, so the difference concerns the action representation and its execution interface. A pose target specifies where the base should move, leaving the controller to determine the velocity needed to reach

Lidded Box Packing



Disassemble and Deliver

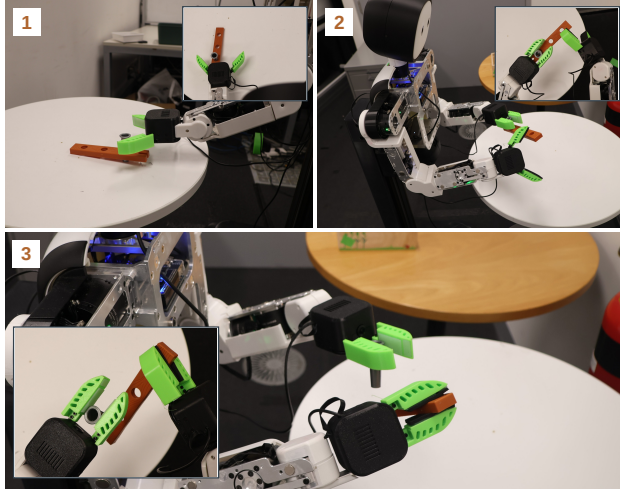


Fig. 7: Numbered execution snapshots for Lidded Box Packing and Disassemble and Deliver.

it. Velocity supervision also reflects demonstration timing, including acceleration, pauses, and the decision to stop; the nearby-state label variation in Fig. 5 illustrates this dependence. This explains the advantage of spatial targets with feedback.

For Q3, the map-frame pose variant with pose input outperforms the variant without it, both trained without augmentation. The output representation is unchanged; the added input locates the base in the target frame, reducing the need to infer its location and heading from vision alone.

C. Robustness to Localisation Noise (Q4)

Experimental setup: Localisation errors affect the base pose supplied to the policy. Table III tests whether pose-noise augmentation improves tolerance to these errors. For each task, we compare map-frame pose policies trained with and without the augmentation in Eq. 6. At each policy inference, we add independent Gaussian noise to the pose input. The tested noise levels and their standard deviations ($\sigma_x = \sigma_y, \sigma_\psi$) are no added noise (0 cm, 0°), low (1 cm, 0.5°), medium (2 cm, 1°), and high (4 cm, 2°). The controller uses the original localisation estimate. Entries report task success rates, with the no-added-noise results taken from Table II.

TABLE III: Success rates (%) under Gaussian pose-input noise, with and without training augmentation.

Policy variant	No added noise	Low	Medium	High
<i>Disassemble and Deliver</i>				
Map-frame pose (w/ pose)	84%	76%	68%	64%
Map-frame pose (w/ pose + aug.)	90%	88%	78%	68%
<i>Lidded Box Packing</i>				
Map-frame pose (w/ pose)	76%	72%	68%	62%
Map-frame pose (w/ pose + aug.)	88%	90%	86%	76%

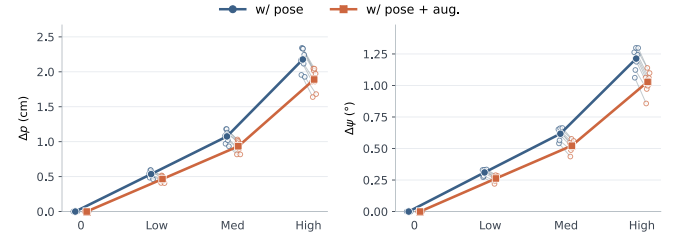


Fig. 8: Offline base-target changes under pose-input noise: Δp is planar displacement and $\Delta\psi$ is wrapped yaw change. Open markers show recording means; solid curves weight recordings equally.

Results: Table III shows that pose-noise augmentation improves success at every tested perturbation level on both tasks. Increasing the perturbation generally makes execution less reliable, but augmentation retains an advantage despite variation between adjacent noise levels. The benefit is therefore not confined to the unperturbed evaluation condition.

This behaviour is consistent with the training objective: noisy pose inputs are paired with unchanged action targets, encouraging appropriate actions under pose noise.

Pose-input sensitivity: To complement Table III, we perturb the map-frame (x, y, ψ) input offline at the same noise levels while holding reconstructed RGB and proprioceptive observations fixed. Both checkpoints receive identical observations and perturbation samples. We measure changes relative to each checkpoint’s own unperturbed base-target prediction at a nominal 0.267 s horizon. Figure 8 shows smaller mean position and yaw changes for *w/ pose + aug.* at every nonzero noise level, consistent with the task-success results.

D. Contribution of Pose-Error Feedback (Q5)

Experimental setup: We compare FF only and FF + FB map-frame pose control on Conveyor Picking (Figure 9). Both use the same checkpoint trained with pose-noise augmentation; FF denotes feedforward and FB denotes pose-error feedback. FF only converts predicted pose-target changes into base velocities; FF + FB corrects the error between the target pose and current localisation estimate.

Metrics: We report task success rate and time-weighted positional/yaw tracking RMSE (planar distance and wrapped yaw error between estimated base pose and active target, using localisation estimates).

Results: Figure 9 shows that adding pose-error feedback improves task success and positional tracking, while yaw tracking changes little. With the policy checkpoint fixed, this gain comes from target execution. Feedforward follows changes in those targets but does not respond to an offset between the target and the estimated base pose. Feedback corrects this offset to reach suitable picking positions.

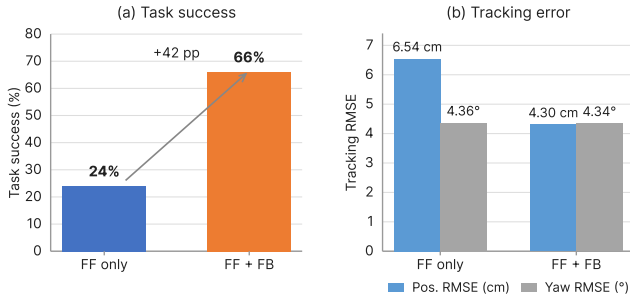


Fig. 9: Feedback ablation on Conveyor Picking: success and time-weighted tracking RMSE over 50 rollouts per variant using one augmented policy checkpoint. Feedback improves success and positional tracking, with little change in yaw RMSE.

TABLE IV: Geometric mapping results across the six manipulation tasks. Deployable pairs require a complete task map, and relative cost includes feature matching and reconstruction.

Variant	Complete maps	Consistent pairs	Deployable pairs	Cost (rel.)
Full mapping pipeline	6/6	911/912	911/912	1.00
Without arm mask	3/6	908/912	600/912	1.58
Temporal-only matching	0/6	892/912	0/912	0.31
Without temporal neighbours	5/6	891/912	780/912	0.94
Nearest-neighbour matching	4/6	878/912	708/912	0.55

E. Geometric Map Construction Ablation (Q6)

Shared protocol: Table IV evaluates the map-construction pipeline across the six manipulation tasks. For each task, the data used for map construction and localisation queries are disjoint. All variants use the same held-out stereo queries, reconstruction budget, calibration, feature extraction, and localisation thresholds. VGGT4D-based dynamic-region masking is retained in all variants. We ablate the remaining geometric components by removing robot-arm masking, restricting matching to temporal neighbours, removing temporal-neighbour matches, or replacing LightGlue with nearest-neighbour matching.

Metrics: A complete map is a single connected reconstruction that registers all selected reference images and passes the geometric quality checks. A stereo query is considered consistent when both views localise independently against the largest reconstructed component and their estimated poses agree within 3cm and 2°. Deployable pairs additionally require the corresponding task map to be complete. Relative cost is the median ratio of feature-matching and reconstruction time to the full mapping pipeline. Stereo consistency measures agreement between the two views rather than absolute pose accuracy.

Results: The full mapping pipeline reconstructs complete maps for all six tasks. Temporal-only matching fails to produce a complete map, while the other ablations also reduce the number of deployable localisation queries. Some queries can still localise consistently within the largest component of an incomplete reconstruction, explaining why consistent-pair counts can remain high even when deployable-pair counts decrease.

F. Policy Architectures (Q7)

Experimental setup: Table V compares unanchored velocity control with map-frame pose control. Within each family,

TABLE V: Success rates for ACT, DP, and FM over 50 rollouts per task. V: unanchored velocity without pose input; P: map-frame pose targets with pose input and localisation feedback.

Task	ACT		DP		FM	
	V	P	V	P	V	P
Disassemble and Deliver	38%	90%	42%	86%	32%	88%
Drawer Packing	20%	64%	44%	62%	38%	66%
Conveyor Picking	42%	66%	38%	74%	36%	76%
Lidded Box Packing	44%	88%	52%	56%	50%	60%
Bag Packing	18%	66%	28%	68%	32%	70%
Dual-Drawer Return	22%	68%	30%	70%	36%	74%

demonstrations, RGB and joint/gripper states, arm/gripper action representations, rollout counts, and success criteria are fixed. Unanchored velocity omits the map-frame base-pose input. Map-frame pose control includes it and tracks pose targets with feedback. ACT’s two configurations match *Unanchored velocity/Map-frame pose (w/ pose + aug.)* in Table II. ACT uses action chunks [27], DP iterative denoising [28], and FM a learned vector field [29].

Results: Table V shows that map-frame pose control improves success across every evaluated task and policy family. The common trend across ACT, DP, and FM supports the use of MAVP with different action-generation mechanisms. Each policy learns shared-map targets tracked using the current pose estimate, whether generated by a chunked transformer, iterative denoising, or a learned flow. This comparison extends the evidence across policy architectures, evaluating pose inputs, map-frame targets, and localisation feedback jointly; preceding ablations examine the individual choices.

G. Ongoing Map Localisation (Q8)

Experimental setup: Table VI compares *initial alignment only* with *ongoing map localisation* on Disassemble and Deliver and Lidded Box Packing. We reuse the same pose-noise-augmented map-frame pose checkpoint and FF + FB controller from the Q1–Q5 ablations; the ongoing-localisation numbers are taken from *Map-frame pose (w/ pose + aug.)* in Table II. We evaluate 50 initial-only trials per task using the corresponding deployment settings and task-completion criteria.

Initial alignment only fixes the local-to-map alignment after the first accepted map localisation in each deployment session and disables subsequent map queries. Local odometry and camera-to-base kinematics continue to update the pose supplied to both the policy and controller. Restarting the policy between trials retains this alignment and odometry state. *Ongoing map localisation* instead updates the alignment using accepted map localisations during execution. Policy timing, action chunks, target lookahead, controller parameters, velocity limits, and task time limits follow the corresponding baseline settings, with no additional localisation noise. A local-tracking reset ends the initial-only session rather than triggering map-based recovery.

Results. Table VI shows that ongoing map localisation achieves higher task success on both tasks. This result indicates that continued localisation helps maintain a consistent spatial reference for the policy and controller. Accepted map updates correct the local-to-map alignment, keeping policy

TABLE VI: Task success (%) with session-level initial alignment retained across policy restarts versus ongoing map localisation. Ongoing-localisation results are reused from Table II.

Task	Initial alignment only	Ongoing map localisation
Disassemble and Deliver	64%	90%
Lidded Box Packing	52%	88%

pose inputs and controller feedback referenced to the same map frame as the predicted targets.

V. CONCLUSION AND FUTURE WORK

We presented MAVP, a map-aware visuomotor framework that gives mobile manipulation policies a shared spatial reference for learning and execution. MAVP reconstructs a map from teleoperated demonstrations, expresses demonstrated and predicted base poses in this common frame, and couples learned base-pose targets with localisation-based feedback control. This formulation separates *where* the base should move from the low-level velocities required to reach that pose, while explicit pose conditioning and pose-noise augmentation improve robustness to spatial uncertainty. Across real-world tasks and multiple visuomotor policy families, the results consistently support shared-map pose targets over unanchored base control and demonstrate the value of feedback and continued localisation during execution. Future work will investigate more scalable map construction and localisation, richer uncertainty-aware policy and control mechanisms, and adaptation across diverse scenes and task configurations. Extending MAVP to larger workspaces, longer-horizon behaviours, and more general whole-body mobile manipulation offers a natural path towards reliable deployment in increasingly complex environments.

REFERENCES

- [1] Z. Fu, T. Z. Zhao, and C. Finn, “Mobile ALOHA: Learning Bimanual Mobile Manipulation using Low-Cost Whole-Body Teleoperation,” in *Proceedings of the 8th Conference on Robot Learning*, 2024.
- [2] J. Yang, I. Huang, B. Vu, M. Bajracharya, R. Antonova, and J. Bohg, “Mobi- π : Mobilizing Your Robot Learning Policy,” in *Proceedings of the 9th Conference on Robot Learning*, 2025.
- [3] P. Sundaresan, R. Malhotra, P. Miao, J. Yang, J. Wu, H. Hu, R. Antonova, F. Engelmann, D. Sadigh, and J. Bohg, “HoMeR: Learning In-the-Wild Mobile Manipulation via Hybrid Imitation and Whole-Body Control,” in *Proceedings of the IEEE International Conference on Robotics and Automation (ICRA)*, 2026.
- [4] J. Wu, X. Sun, A. Zeng, S. Song, J. Lee, S. Rusinkiewicz, and T. Funkhouser, “Spatial action maps for mobile manipulation,” in *Proceedings of Robotics: Science and Systems (RSS)*, 2020.
- [5] S. Kim, W. Chung, Z. Dai, D. Bhatt, A. Shukla, H. Su, Y. Tian, and N. Atanasov, “Seeing the Bigger Picture: 3D Latent Mapping for Mobile Manipulation Policy Learning,” in *Proceedings of the IEEE International Conference on Robotics and Automation (ICRA)*, 2026.
- [6] J. Gu, D. S. Chaplot, H. Su, and J. Malik, “Multi-skill mobile manipulation for object rearrangement,” in *International Conference on Learning Representations (ICLR)*, 2023.
- [7] H. Ha, Y. Gao, Z. Fu, J. Tan, and S. Song, “UMI-on-Legs: Making Manipulation Policies Mobile with Manipulation-Centric Whole-body Controllers,” in *Proceedings of the 8th Conference on Robot Learning*, 2025.
- [8] X. Xu, J. Park, H. Zhang, E. Cousineau, A. Bhat, J. Barreiros, D. Wang, J. Bohg, and S. Song, “HoMMI: Learning Whole-Body Mobile Manipulation from Human Demonstrations,” in *Proceedings of Robotics: Science and Systems (RSS)*, 2026.
- [9] Q. Sun, W. Zhi, T. Zhang, and M. Johnson-Roberson, “Teaching Robots Where To Go And How To Act With Human Sketches via Spatial Diagrammatic Instructions,” in *Proceedings of the IEEE/RSSJ International Conference on Intelligent Robots and Systems (IROS)*, 2024, pp. 1094–1100.
- [10] Y. Ze, G. Zhang, K. Zhang, C. Hu, M. Wang, and H. Xu, “3D diffusion policy: Generalizable visuomotor policy learning via simple 3D representations,” in *Proceedings of Robotics: Science and Systems (RSS)*, 2024.
- [11] L. X. Shi, A. Sharma, T. Z. Zhao, and C. Finn, “Waypoint-Based Imitation Learning for Robotic Manipulation,” in *Proceedings of the 7th Conference on Robot Learning*, 2023.
- [12] S. Belkhale, Y. Cui, and D. Sadigh, “HYDRA: Hybrid Robot Actions for Imitation Learning,” in *Proceedings of the 7th Conference on Robot Learning*, 2023.
- [13] S. Ross, G. Gordon, and D. Bagnell, “A Reduction of Imitation Learning and Structured Prediction to No-Regret Online Learning,” in *Proceedings of the Fourteenth International Conference on Artificial Intelligence and Statistics (AISTATS)*, 2011.
- [14] M. Laskey, J. Lee, R. Fox, A. Dragan, and K. Goldberg, “DART: Noise Injection for Robust Imitation Learning,” in *Proceedings of the 1st Conference on Robot Learning (CoRL)*, 2017.
- [15] J. L. Schönberger and J.-M. Frahm, “Structure-from-Motion Revisited,” in *Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition (CVPR)*, 2016, pp. 4104–4113.
- [16] P.-E. Sarlin, C. Cadena, R. Siegwart, and M. Dymczyk, “From Coarse to Fine: Robust Hierarchical Localization at Large Scale,” in *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR)*, 2019, pp. 12 716–12 725.
- [17] B. Bescos, J. M. Fàcil, J. Civera, and J. Neira, “DynaSLAM: Tracking, Mapping, and inpainting in Dynamic Scenes,” *IEEE Robotics and Automation Letters*, vol. 3, no. 4, pp. 4076–4083, 2018.
- [18] L. Goli, S. Sabour, M. Matthews *et al.*, “RoMo: Robust Motion Segmentation Improves Structure from Motion,” in *Proceedings of the IEEE/CVF International Conference on Computer Vision (ICCV)*, 2025.
- [19] Y. Hu, C. Cheng, S. Yu, X. Guo, and H. Wang, “VGGT4D: Mining Motion Cues in Visual Geometry Transformers for 4D Scene Reconstruction,” in *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR)*, Jun. 2026, pp. 414–424.
- [20] P.-E. Sarlin, D. DeTone, T. Malisiewicz, and A. Rabinovich, “SuperGlue: Learning Feature Matching with Graph Neural Networks,” in *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR)*, 2020.
- [21] P. Lindenberger, P.-E. Sarlin, and M. Pollefeys, “LightGlue: Local Feature Matching at Light Speed,” in *Proceedings of the IEEE/CVF International Conference on Computer Vision (ICCV)*, 2023, pp. 17 627–17 638.
- [22] W. Zhi, H. Tang, T. Zhang, and M. Johnson-Roberson, “Unifying Representation and Calibration With 3D Foundation Models,” *IEEE Robotics and Automation Letters*, 2024.
- [23] H. Tang, T. Zhang, M. Johnson-Roberson, and W. Zhi, “Bi-Manual Joint Camera Calibration and Scene Representation,” in *Proceedings of the IEEE International Conference on Robotics and Automation (ICRA)*, 2026.
- [24] D. DeTone, T. Malisiewicz, and A. Rabinovich, “SuperPoint: Self-Supervised Interest Point Detection and Description,” in *Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition (CVPR) Workshops*, 2018, pp. 224–236.
- [25] A. Korovko, D. Slepichev, A. Eftorov, A. Dzhumamuratova, V. Kuznetsov, H. Rabeti, J. Biswas, and S. Pouya, “cuVSLAM: CUDA Accelerated Visual Odometry and Mapping,” arXiv preprint arXiv:2506.04359, 2025.
- [26] W. Peebles and S. Xie, “Scalable Diffusion Models with Transformers,” in *Proceedings of the IEEE/CVF International Conference on Computer Vision (ICCV)*, 2023.
- [27] T. Z. Zhao, V. Kumar, S. Levine, and C. Finn, “Learning Fine-Grained Bimanual Manipulation with Low-Cost Hardware,” in *Proceedings of Robotics: Science and Systems (RSS)*, 2023.
- [28] C. Chi, S. Feng, Y. Du, Z. Xu, E. Cousineau, B. Burchfiel, and S. Song, “Diffusion Policy: Visuomotor Policy Learning via Action Diffusion,” in *Proceedings of Robotics: Science and Systems (RSS)*, 2023.
- [29] Y. Lipman, R. T. Q. Chen, H. Ben-Hamu, M. Nickel, and M. Le, “Flow Matching for Generative Modeling,” arXiv preprint arXiv:2210.02747, 2022.